KDL OS: Vision vs Code Gap Report

Kalam Dream Labs

November 7, 2026

CONTENTS

KDL OS: Vision vs Code Gap Report

1. Scorecard

2. What already exists (strengths)

3. User stories in detail

    Story 1. Admin panel only: PARTIAL

    Story 2. Admin + Frontend website: PARTIAL

    Story 3. Static website: PARTIAL

    Story 4. Landing pages: PARTIAL (export NO)

    Story 5. Multiple projects: NO

4. Technical requirements in detail

    A. Module management: PARTIAL

    B. Database and seeders: PARTIAL

    C. Section Builder: YES (mostly)

    D. Theme and template management: PARTIAL

    E. Export: PARTIAL

5. Install, onboarding and docs gaps

6. Prioritized gap list

    P0. Blocks the vision

CONFIDENTIAL

P1. Experience gaps

P2. Module system maturity

P3. Documentation

7. Security items that touch these gaps

8. Open questions for you

CONFIDENTIAL

KDL OS: Vision vs Code Gap Report

Date: 2026-11-07. Branch: akhil_kdl (HEAD ad454b8e). Method: four read-only code audits against the vision document (5 user stories, requirements A to E). Nothing was run: no Docker, no seed, no browser. Findings are from reading code, plus the existing build output in `frontend/projects/default/out`. Items marked “unverified” need a live check.

Verdict key: YES = built and matches the vision. PARTIAL = built but incomplete or differs. NO = missing.

CONFIDENTIAL

1. Scorecard

Area	Verdict	One-line reason
Story 1. Admin panel only	PARTIAL	Works. Missing Light/Dark/System setting UI, separate company vs app name and logo, Admin and Developer users
Story 2. Admin + Frontend website	PARTIAL	Modules, nav and routes activate. Demo content is opt-in, no guided workflow, no project-level launch
Story 3. Static website	PARTIAL	Build flow works. Detail pages, per-page SEO, sitemap missing. Not backend-independent
Story 4. Landing pages	PARTIAL, export is NO	Create, list, edit, templates work. No Launch button, no `landingpages` folder, no export
Story 5. Multiple projects	NO	No project type, no create-project UI, theme and slugs are global
A. Module management	PARTIAL	Runtime for built-in modules only. No update flow, uninstall blocked
B. Database and seeders	PARTIAL	Templates and themes are code, not DB rows. Demo is not automatic
C. Section Builder	YES (mostly)	Full feature set. No page duplicate, no project switcher
D. Theme and template management	PARTIAL	Theme is global, not per project
E. Static and landing export	PARTIAL	See Story 3 and 4

CONFIDENTIAL

2. What already exists (strengths)

- Module system: manifest (`module.json`, Zod-validated), install, enable, disable, dependency cascade, conflict handling, RBAC permission registration, dynamic sidebar. Toggling a non-core module takes effect at once with no restart (`moduleGate`, 404 when disabled).
- 36 module manifests: 15 core (always on) and 21 optional.
- Products flow: Admin, Website, Landing Pages. The Setup Wizard enables the right modules.
- Setup Wizard: company identity, logo, favicon, contact details, products, optional starter site (Home, About, Services, Contact, FAQ). Skippable.
- Section Builder: predefined blocks (general, landing, construction, medical), no-code composer, edit, duplicate, delete, reuse, device visibility, saved in DB (`CustomBlockTemplate`).
- Page builder (Puck): mobile 390, tablet 768, desktop 1280 viewports, versions, restore, publish, preview and share tokens.
- Template Engine Studio: brand intake, palette from logo, approval, website generation, layout (header, top bar, footer variants), menus.
- Identity settings: app name, logos, favicon, primary and secondary colors, contact, social. Colors apply app-wide from one source.
- Landing module: 5 templates, multiple landing pages per project, per-landing form.
- Static site build: `output:'export'`, per-project folder, zip download, status polling.
- RBAC: Super Admin, Admin, Developer, User roles with blueprints.
- Admin-only sidebar is clean on a fresh install (only core nav).

CONFIDENTIAL

3. User stories in detail

Story 1. Admin panel only: PARTIAL

Vision step	Status	Detail
Clone, install, configure env and DB, run	PARTIAL	Works with `docs/INSTALLATION_GUIDE_SIMPLE.md`. No one-command setup. Seed is a separate manual command
Log in as Super Admin, Admin or Developer	NO	Seed creates only the Super Admin (`backend/prisma/seed.js:16–35`). Admin and Developer exist as roles with no users
Modules section: install only admin modules	YES	`/admin/modules`, plus Products page. Admin product = users, user-management, settings, identity, media
Company name, company logo	PARTIAL	One `app_name` field serves both company and application name
Application name, application logo	PARTIAL	No separate application logo. Only `logo_light`, `logo_dark`, `favicon`
Theme Light, Dark or System	NO on admin-only	Lives in the Theme Engine UI (`theme-engine-ui`, not core). Not installed in an admin-only setup. Not part of Identity
Primary and secondary colors	YES	`brand_primary`, `brand_secondary` in Identity
No frontend module or navigation	YES	Verified in nav code. Not verified in a browser

CONFIDENTIAL

Story 2. Admin + Frontend website: PARTIAL

Vision step	Status	Detail
Install Frontend module	YES	The `cms` umbrella installs faq, gallery, testimonials, team, catalog, work, sectors, projects-content, menus, landing, forms
Auto-enable routes and nav	YES	Runtime, via moduleGate and `/api/modules/enabled`
Auto-create demo project	PARTIAL	Not automatic. Needs the wizard answer `onboarding.frontend=yes`. A plain Modules-page install gives no pages. No separate “demo project” is created, everything lands in the Default Project
Load sections, templates, sample pages, dummy content	PARTIAL	Pages and sample rows are created from code constants. No template library table. Dependency seeds insert DemoLogo rows regardless of the wizard answer
Edit, replace, delete demo content	YES	Seeded rows have no lock flag. Page duplicate endpoint is missing
Frontend workflow (11 steps)	PARTIAL	Steps exist as scattered screens. No single guided wizard in vision order
Preview on desktop, tablet, mobile	PARTIAL	Per page in the Puck editor. No project-level preview
Launch and publish	PARTIAL	Per-page Publish only. No project-level Launch (no go-live, domain or deploy step)

Workflow step map:

CONFIDENTIAL

Vision step	Existing screen
1 Project Setup	Template Engine Overview (intake), Settings > Identity. No create-project, no type
2 Theme	`/admin/theme-engine`, `/admin/color-palette`, `/admin/brands` (global)
3 Homepage	`/admin/pages` and the page editor. No dedicated homepage step
4 Header and Navigation	`/admin/menus`, Website > Navigation step
5 Footer	`/admin/layouts`, `.../website/layout`
6 Page Management	`/admin/pages`
7 Section Builder	`/admin/page-builder/section-builder`
8 Layout	`/admin/layouts`
9 Content	Team, FAQ, Sectors, Catalog, Work, Gallery, Testimonials, Case Studies (default project only)
10 Preview	Puck viewports, preview and share tokens
11 Launch	Per-page publish only

CONFIDENTIAL

Story 3. Static website: PARTIAL

Requirement	Status	Detail
Generate button	YES	Labelled “Convert to Static Website” (`admin/static-website/page.tsx`). Flow: write site folder from DB, `npm install` , `next build` with `output:'export'` , zip
Separate folder, complete site	PARTIAL	`frontend/projects/<slug>/out` , replaced on every build. The parent folder also holds node_modules, src, .next
All configured pages and URLs	PARTIAL	Normal pages yes. Work, catalog and sector detail pages are deleted from the export (`build-static.mjs:39-41`). Draft pages are exported (no status filter). Landing pages can leak in (no `site_id` filter, code read only)
Works with no KDL OS, DB, backend, Node	PARTIAL	`http://localhost:4000/api` is baked into the JS. Media hotlinks MinIO, including a favicon with a 7-day presigned URL. Contact form works only with a live `apiBase` , else it becomes a mailto and tel card. No MinIO-to-local asset rewrite exists
Responsive, navigation, animations	YES	Menus and header and footer are baked from `menus.json` . Fonts self-hosted (needs Google at build time)
SEO	NO	Per-page SEO never selected. Same meta description on every page. No canonical, OG or Twitter tags. No `sitemap.xml` , no `robots.txt`
No admin or backend files in export	PARTIAL	Import-closure copy, regex-based. No test asserts the output is free of admin code
Regenerate after edits	PARTIAL	Repeatable by clicking again. Auto-resync does not rebuild `out/` , so it goes stale
Status and error messages	PARTIAL	States idle, running, done, failed in memory only. After a backend restart the status is idle and the zip returns 404 although `out/` exists. Generic error plus a 40-line log tail. No timeout on spawned processes
Zip download	PARTIAL	Built in memory (AdmZip). Every project downloads as `static-site.zip`

CONFIDENTIAL

Requirement	Status	Detail
One project never overwrites another	YES	Folder keyed by slug, path-escape guard
Permissions	PARTIAL	UI page needs `menus:view`, API needs `template-engine:run` and `:export` (mismatch)

CONFIDENTIAL

Story 4. Landing pages: PARTIAL (export NO)

Requirement	Status	Detail
Landing Pages section, create project	YES	`/admin/landing-pages`. Each landing page is a `Site` of type LANDING
Predefined templates	YES	5 templates, code constants in `landing/templates.js`
Build from Section Builder	YES	Landing pages are page-builder pages
Customize theme and colors	PARTIAL	Global theme only. Branding dialog edits site name, description, contact
Header and footer	PARTIAL	Branding dialog only
Forms and buttons	YES	Form created per template that has one
SEO settings	PARTIAL	Generic page-builder `seo` field only, no landing-specific settings
Preview	PARTIAL	Template preview gallery and live link `/p/<slug>` (needs backend and published page)
Launch / Generate Landing Page button	NO	
`landingpages` output folder	NO	Zero hits in the repo
Independent files and URL per landing page	NO	Served dynamically at `/p/<slug>`. A `domain` field exists on Site, host serving not traced
Static export, regenerate	NO	No export endpoint in the landing module
Multiple landing pages	YES	

CONFIDENTIAL

Story 5. Multiple projects: NO

Requirement	Status	Detail
Project has name, slug, type, config, status	NO	`Project` has name, slug, is_default, is_shared only (`projects.prisma:6–27`). No type, status, config. Header still says “minimal stub”
Create multiple projects from the UI	NO	`POST /api/projects` exists. The frontend never calls it. No `/admin/projects` route
Project types (admin, website, landing, static)	NO	Only `SiteType { WEBSITE, LANDING }` on Site. No ADMIN or STATIC
Independent theme, colors, nav, layouts, pages, sections, content	PARTIAL	See isolation table below
Per-project preview and launch	NO	
Editing one project never affects another	NO	Theme, identity and media are global
Central dashboard of all projects	NO	Dashboard never queries `/projects`
Combinations (A to G)	NO	One default project assumed everywhere: landing, onboarding, cms, createPage, content screens, Template Engine redirect

Isolation table:

CONFIDENTIAL

Project-scoped (has `project_id`)	**Global (no `project_id`)**
Site, SiteLayout, Menu (nullable), CustomBlockTemplate, Form, DetailPageType and Entry, BrandKit, CollateralAsset, TemplateEngineRun, credits tables, ProjectSettingValue	Theme engine tokens (`SettingValue` keyed by platform), active theme mode, button and form styles
BuilderPage (nullable)	Application Identity (one company per install; only a per-Site override exists)
Content tables with NULLABLE `project_id`: TeamMember, FaqEntry, Sector, CatalogItem, WorkCategory, GalleryItem, Testimonial, ProjectCaseStudy	Media and DAM tables
	Modules enabled state (per install)
	`BuilderPage.slug` is globally `@unique` and `listPages()` has no project filter

CONFIDENTIAL

4. Technical requirements in detail

A. Module management: PARTIAL

Requirement	Status	Detail
Predefined built-in modules	YES	36 manifests (STATUS.md says 28, stale)
Install, enable, disable	YES	Dependency cascade, conflicts, 409 when a dependant is enabled
Configure	PARTIAL	Raw JSON patch only (`Module.settings`). Nothing consumes it as a typed schema
Auto-activate routes, permissions, nav, settings	YES for built-in, NO for new modules	A new module folder needs a backend restart. Its admin page is a static Next route needing a frontend rebuild. Prisma tables come from `migrate deploy` at container start, not at install
Disable hides or deactivates without affecting others	PARTIAL	Real server gate for the 21 non-core modules. Core engines (page-builder, theme-engine, custom-blocks, projects) have no gate. Public page-builder routes stay live when `cms` or `landing` is disabled. `ModuleGuard` is UI only
Frontend and admin files organized separately	PARTIAL	Clean split in the frontend app (`app/admin` , `app/(site)` , `admin/` , `frontend/`). Backend is a flat module-per-folder layout, admin and public endpoints share one router
No module upload, ZIP or marketplace	NO	A module must already be on disk. Scaffold: `npm run module:create`
Module update flow	NO	`Module.version` is written once, never compared with the manifest. Seeds do not re-run on upgrade
Uninstall	PARTIAL	`deregisterPermissions` throws 409 while roles hold the permission, and install auto-grants them to Developer and Admin. Only theme-engine and template-engine ship `uninstall.js`
Cache invalidation	LOW RISK	`invalidateModuleCache` has no try/catch around `redis.del`

CONFIDENTIAL

B. Database and seeders: PARTIAL

Requirement	Status	Detail
Templates, themes, demo projects, pages, sections database-driven	PARTIAL	Landing templates, starter pages and CMS pages are code constants written to DB on demand. No Template model and no theme-preset library. Section schemas come from a JSON file. Block packs are frontend code. Only user-created custom blocks are DB rows
Seeders populate at install	YES	Per-module `seed.js` runs inside the install transaction. Global `prisma/seed.js` is manual
Edit, duplicate, customize, delete predefined content	PARTIAL	Edit and delete yes. No page duplicate endpoint. Sections can be duplicated
No hardcoded editable content in frontend components	PARTIAL	Block pack defaults and starter bodies live in code
Frontend install auto-creates demo project	PARTIAL	See Story 2
Reinstall or update must not duplicate or overwrite	PARTIAL	Natural-key dedupe works. Uninstall and reinstall brings back demo rows the user deleted, and duplicates ones the user renamed. `seed-demo-site.js` replaces page content by slug and would overwrite user edits (manual script)
Seed correctness issues		`menus/seed.js` links to `/p/demo-*` pages that exist only after the manual `seed-demo-site.js`, so they are dead links on a clean install. Dependency seeds insert DemoLogo rows for faq, team, projects-content regardless of the wizard answer. `seed-BrandKit` runs although brand-kit is not installed. `work`, `sectors`, `catalog`, `gallery`, `testimonials` seeds are stubs or no-ops

CONFIDENTIAL

C. Section Builder: YES (mostly)

Present: predefined designs, create without code (composer atoms: columns, padding, background color and image, device visibility), edit, duplicate, delete, usage tracking, reuse across pages and layouts, reorder, responsive viewports, saved in DB. Gaps: no page duplicate endpoint, no per-project switcher (uses default project or `?projectId`), `page-builder-ui` and `theme-engine-ui` are nav stubs.

D. Theme and template management: PARTIAL

Present: predefined token themes (dark, light, focus, all), 12 button presets, brand-kit palette from logo, custom tokens with version history, Light/Dark/System as a platform setting, header and footer variants per project via SiteLayout, theme previews. Gaps: theme and active mode are global, not per project. No project-wide preview. Reusable page layouts exist only for detail pages.

E. Export: PARTIAL

See Story 3 and 4. Also: one project's export never overwrites another's (YES), repeated generation (PARTIAL), clear status and errors (PARTIAL).

CONFIDENTIAL

5. Install, onboarding and docs gaps

Item	Finding
Fresh clone	Works through the simple guide. `docker-compose.override.yml` is gitignored (good, it remaps ports)
README quick start	Not enough alone: `.env.example` lacks `APP_ENCRYPTION_KEY`, leaves MinIO and Meili as `change_this_...`, and its URLs differ from what the simple guide generates
`setup.sh`	Outdated: Paperclip wording, needs OpenRouter, Sentry, Gmail credentials, starts only the infra stack, never seeds
Seeded accounts	Super Admin only
Wizard	Shown only to Super Admin and Developer. “Later” is stored in browser localStorage. A code comment says the landing module does not exist (stale)
`STATUS.md`	Stale: 28 modules, real count 36. Missing cms, catalog, forms, gallery, identity, landing, testimonials, work, detail-page-types

Docs contradictions to fix:

1. `docs/ADMIN_ACCESS.md` says `Admin@123` no longer works. The reset script accepts it outside production (8 character minimum), so README and the simple guide are right.
2. `docs/ADMIN_ACCESS.md` and `docs/SETUP.md` say the reset script needs 12 characters. Code: 8 outside production, 12 in production.
3. `docs/SETUP.md` ports: frontend 3000, MinIO 9000/9001, ai-services 5000. Docker stack: 3001, 9002/9003, 5001.
4. README credentials table: PostgreSQL `postgres/postgres` and MinIO `minioadmin/minioadmin`. The guides generate random passwords and MinIO user `kdl_minio`.
5. README and CLAUDE.md say Prisma 6. `backend/package.json` has Prisma 7.10.
6. README and CLAUDE.md say pnpm monorepo. There is no root `package.json` or `pnpm-workspace.yaml`.
7. `kdl-starter-kit/CLAUDE.md` says “Phase 1 in progress” and 6 core modules. Repo has about 36.
8. The simple guide describes an old “Do you need a front-end website?” popup. Real UI is a 2-step wizard.

CONFIDENTIAL

9. README says modules are under Settings > Products and Modules. Modules is its own sidebar item.
10. `docs/TEMPLATE_ENGINE_FLOW.md` describes a 27-platform “Select Platform” step that is not in the code, and a “Export” stage that is a manifest, unrelated to static export.
11. `.env.example` still has OpenRouter and Anthropic text that conflicts with the “OpenRouter retired” note in CLAUDE.md.

CONFIDENTIAL

6. Prioritized gap list

Effort is a rough guess for planning, not a commitment.

P0. Blocks the vision

ID	Gap	Rough effort
G1	Project model: add type, status, config. Project list, create and switch UI, central dashboard	L
G2	Remove the “default project” assumption (landing, onboarding, cms, createPage, content screens, Studio redirect). Make `project_id` required on content tables	L
G3	Per-project theme and identity (tokens, active mode, button styles, media scope)	L
G4	`BuilderPage.slug` unique per project, `listPages` filtered by project	M
G5	Landing page Launch and Generate button, `landingpages` output folder, per-landing static export	L
G6	Static export fixes: detail pages, per-page SEO, sitemap and robots, local assets (replace MinIO URLs), no baked `localhost:4000`, working forms without KDL OS, exclude drafts and landing pages	L

CONFIDENTIAL

P1. Experience gaps

ID	Gap	Rough effort
G7	Guided frontend workflow (11 steps) with project-level preview and Launch	L
G8	Demo project created automatically on Frontend install, as its own project. Template and theme library as DB rows	M
G9	Light/Dark/System setting available on admin-only installs. Separate company and application name and logo	S
G10	Seed Admin and Developer users (or a clear create-user step). One-command setup	S
G11	Static build robustness: persist job state, timeouts, clear errors, per-project zip name, versioned exports	M
G12	Page duplicate endpoint	S

P2. Module system maturity

ID	Gap	Rough effort
G13	Gate core engines and public page-builder routes when their product is disabled	M
G14	Module version update flow and seed re-run hook. Fix uninstall permission block. Typed module settings	M
G15	Seed fixes: dead `/p/demo-*` menu links, dependency seeds ignoring the wizard answer, reinstall resurrecting deleted demo rows	S

P3. Documentation

ID	Gap	Rough effort
G16	Fix the 11 doc contradictions, refresh `STATUS.md`, retire or rewrite `setup.sh`	S

CONFIDENTIAL

7. Security items that touch these gaps

These came from the earlier security reviews and overlap with the work above. Details are in the six review reports from this session.

- Static export build runs `npm install` with the full API environment (High). Isolate it when reworking G6 and G11.
- Static build and download skip the approval gate and are not audit-logged. `apiBase` accepts any URL scheme.
- A database dump with password hashes is committed in `backups/` (release blocker, unrelated to the vision but urgent).
- Production nginx has no TLS or HSTS, and prod inherits the dev `site-preview` (`next dev`) service.
- Claude spend has no cap in ai-services.

CONFIDENTIAL

8. Open questions for you

1. Should “project type” be one value per project (admin, website, landing, static), or a set of flags so one project can combine them (your Project F and G rows)?
2. Should a landing page be its own Project, or stay a Site inside a Project?
3. Should the static export be built inside the API container (today), or by a separate secret-free worker?
4. Which gap do you want first? The suggested order is G1 and G2 together, then G3 and G4, then G5 and G6.

CONFIDENTIAL